

CYBERDRISHTI

SYSTEM ARCHITECTURE & TECHNICAL DESIGN DOCUMENT

AI-Based Network Attack Forecasting from Network Traffic Data

Problem Statement ID	SIH26153
Problem Statement	AI-Based Network Attack Forecasting from Network Traffic Data
Team	GNC Nivaran
Project	CyberDrishti
Document Type	Architecture Document

1. Purpose and Architectural Objective

CyberDrishti is designed to help defenders look beyond an attack that has already been detected and estimate how it may progress next. Instead of treating network flows as unrelated events, the system preserves their time order and converts them into a sequence of changing network states. A temporal model then learns these transitions and produces a multi-step forecast with probability, confidence and supporting evidence.

2. High-Level Architecture

The system is organized into eight logical layers: (1) ingestion, (2) feature extraction, (3) temporal representation, (4) network-state construction, (5) temporal world model, (6) K-step forecast engine, (7) intelligence layer, and (8) presentation layer.

Layer	Responsibility	Main output
1. Ingestion	Accept CSV, PCAP/PCAPNG, flow records and supported telemetry.	Normalized records
2. Feature extraction	Derive flow- and packet-level behavioural features.	Feature records
3. Temporal representation	Clean, normalize, order and group records into windows.	Ordered windows
4. State builder	Aggregate each window into a network-state vector.	S1 ... St
5. World model	Learn temporal state-transition behaviour using LSTM.	Next-state probabilities
6. Forecast engine	Perform K-step rollout and risk estimation.	Trajectory + infiltration probability
7. Intelligence	Add attack-stage context, evidence and explainability.	Interpretable forecast
8. Presentation	Expose results through API and SOC dashboard.	Analyst intelligence

3. Data Ingestion and Feature Extraction

The prototype works with historical or replayable **CSV/flow records and PCAP/PCAPNG files**. The ingestion layer validates the input, extracts timestamps and converts the source into a consistent format. Invalid or unsupported data is rejected before analysis, while sensitive traffic can be processed locally.

The system uses both flow-level and packet-level information. Flow features include IP addresses and ports, protocol, TCP flags, packets, bytes, duration, inter-arrival statistics and traffic ratios. Packet-level features include TTL, TCP window behaviour, fragmentation, payload-size patterns, retransmissions, timing and scanning-related behaviour. Together, these features capture both communication patterns and lower-level changes in traffic behaviour.

4. Temporal Processing & Network-State Construction

After feature extraction, the records are cleaned, normalized and ordered by timestamp. Maintaining this order is important because shuffling traffic can break the real attack sequence and introduce future-information leakage. Traffic is divided into configurable windows such as **10, 30 or 60 seconds**, and each window is summarized as a network state.

A state can include traffic volume, active flows, unique endpoints, port diversity, protocol distribution, TCP behaviour, connection failures, retransmissions and scan indicators. A sequence can then be represented as:

$[S_1, S_2, S_3, S_4] \rightarrow S_5$

Attack-stage targets are created from the labels and timelines supported by the selected datasets. Possible stages include Normal, Reconnaissance, Initial Access, Lateral Movement, Command and Control, and Exfiltration.

5. Temporal World Model

The primary model is an LSTM implemented with PyTorch. It receives a sequence of recent network states rather than isolated flow records and learns temporal transition dynamics.

Conceptually, the forecasting problem is represented as $P(S(t+1) | S(t-n), \dots, S(t))$. The model learns patterns such as increasing port diversity, repeated connection attempts, timing changes and other evolving network behaviours.

a) Primary model: LSTM. Baseline: Logistic Regression. GRU and Transformer models can be evaluated later. Graph/temporal-GNN approaches are future extensions after the vector-based temporal pipeline is stable.

Model artifacts, preprocessing parameters and configuration should be versioned for reproducibility.

6. K-Step Forecasting Engine

The model first estimates the next state and can then use that predicted state to continue forecasting for **K future steps**. This gives CyberDrishiti a possible attack trajectory instead of only a single label.

For example: **Reconnaissance → Initial Access → Lateral Movement → Command & Control**

The forecast can include the current stage, predicted next stage, future trajectory, infiltration/progression probability and confidence or uncertainty. These are probabilities, not guarantees.

7. Attack Intelligence & MITRE ATT&CK

The intelligence layer translates model outputs into analyst-readable attack-stage information. MITRE ATT&CK is a contextual mapping layer, not the forecasting model itself.

The system keeps Observed, Current and Forecast semantics must remain distinct. A forecasted technique should not be displayed as a confirmed event. Each mapping should have supporting evidence and unsupported technique identifiers should not be invented.

8. Forecast Outputs & Evaluation

The main forecast outputs are the **current stage, confidence, predicted next stage, next-stage probability, K-step trajectory, infiltration probability and uncertainty information**. These results are structured for use by the dashboard and analysis interface.

Evaluation focuses on whether the system can actually anticipate future behaviour. The Logistic Regression baseline provides a comparison point. The main measures include **precision, recall, F1-score, false-positive rate, next-stage accuracy, K-step forecasting performance, Early Warning Time and Brier score/reliability analysis** where applicable. Held-out scenarios or datasets can be used to examine generalization.

9. Technology Stack, Security & Deployment

The planned stack uses **React, TypeScript, Vite, Tailwind CSS and Recharts** for the frontend, **Python and FastAPI** for the application interface, **Scapy/PyShark** for network processing, **Pandas, NumPy and scikit-learn** for data handling, and **PyTorch with LSTM and Logistic Regression** for modelling. SHAP supports explainability, while MITRE ATT&CK provides attack context. The main datasets are **CSE-CIC-IDS2018 and CTU-13**. Docker and local/offline execution are planned for reproducible deployment. Development follows a staged approach: build the traffic and feature pipeline first, create temporal states, establish the baseline and LSTM model, implement K-step forecasting and evaluation, then connect intelligence/XAI, the FastAPI layer and deployment. For security, sensitive traffic should be processed locally where possible, uploaded data should be validated, unnecessary raw-payload exposure should be avoided, and model/preprocessing versions should be kept consistent.

10. Limitations & Architectural Boundary

CyberDrishiti produces **probabilistic forecasts, not guaranteed outcomes**. Its performance depends on the quality and temporal coverage of labelled data, the selected features and how well the model generalizes beyond the training scenarios. Class imbalance and rare attack stages can also affect probability estimates, while uncertainty may increase as the forecast horizon becomes longer. The prototype therefore keeps evidence, uncertainty and forecast status visible to the analyst and does not perform autonomous destructive blocking or isolation. Real-time streaming, SIEM/SOC integration, additional datasets and alternative temporal models remain future extensions.